RunCPM 7xtgv-terminal Reference Guide

Table of Contents

1. Terminal operation
   - Starting the PC mirror
   - Global terminal keys
2. Applications and input
   - Application profiles
   - Histories, banks, and the active line
   - Editing details
   - Our installed application controls
   - RAW input
3. CP/M commands and files
   - 7XF shell and user areas
   - Shared A0 commands
   - Boot, AUTOEXEC, and pending SUBMIT work
   - VTPUT / VTGET from the mirror
   - ZIP transfer and extraction
4. Configuration
   - INI sections and loading
   - Keyboard overrides in `7XTGVTRM.INI`
   - Global settings
   - Display and per-bank geometry
   - Additional bank settings
   - Palette definitions and selection
5. Terminal interfaces
   - Output interpretation modes
   - Implemented escape-sequence subset
   - Text attributes and color interpretation
   - Terminal control and query map
   - Graphics extension: xterm256 plotting

Source references and implementation boundaries
Appendix A. Diagnostics, tests and demonstrations
CARDMARK raw SD read diagnostic
Visual tests in 7xtgv(C1) and DEC graphics
Palette comparison example
Graphics and timing helpers in xterm256(C2)
Plotting demonstrations

This is the exact user-visible behavior of the current RunCPM 7xtgv-terminal, not the proposed native 7xtgv interface. For tasks start with the User Guide; for code and private protocols use the Technical Reference Guide. Settings in the live device INI may differ from compiled defaults and from the local copy. The Features Timeline tracks command and file name changes; syntax below is the current reference.

CP/M commands, terminal profiles, INI settings and output escape sequences are shared by the browser, physical boards and native Linux/Windows desktops. Input delivery, host commands and card lifetime differ by route. For first use, see the browser introduction; for native setup, use the Build Guide.

CPU speed: `SPEED 65535` selects the fastest throttled ("slow") setting; `SPEED 0` restores unthrottled execution, the default. Smaller nonzero values slow the emulator further. A reported i7-10710U (1.1 GHz) test gave roughly 44 MHz in VTCPU with `SPEED 65535`, on both desktop and web. That is an observed equivalent Z80 benchmark rate, not a fixed MHz setting. See the SPEED reference.

The saved PDF uses our **mobile PDF (mpdf)** profile: Legal portrait, 140% scale, zero left/right margins and 0.1-inch top/bottom margins, with backgrounds. It is intended for reading on phones and comfortably enlarged monitor text. For physical printing, render the AsciiDoc with A4/Letter, normal printer margins and a smaller scale. See the guide index for the shared profile and regeneration workflow.

Browser and native input routes

Operation	Standalone browser	Native mirror (hardware or SDL desktop)
Where to type	Click the display or **Keyboard → display**.	Type into the mirror; SDL is output-only.
Ctrl]	Ordinary guest input; focus stays on the display. Click or tap a page control to leave display input.	Introduces the prefix commands below.
Application exit	Alt Q and Alt Esc are encoded when delivered. The Intel MacBook Air check found only Alt Esc working.	Alt Q; Konsole/Kitty also supply Alt Esc. Windows reserves the latter.
Transfer panel / Ctrl Shift X	No native host panel or VTPUT/VTGET host service; use card ZIP controls.	Panel available at an idle CCP prompt.
GCLS shortcut / Pause prefix	No explicit browser mapping for Alt Shift Backspace or Pause. Use the guest graphics helpers/scripts.	Supplied keymaps/prefix commands provide these controls.
Card changes	Memory copy; save in the guest, Stop, then download the ZIP.	Writes directly to the selected host or physical card.

The browser sends existing terminal key events; it does not reproduce the native host bridge. A key in the tables below requires the browser/OS to deliver it. Browser F11 is left to the host. See MacBook observations and compact-keyboard notes, including the unsuccessful Alt Space check. Native prefix commands and transfer shortcuts below are explicitly native-only.



Windows: Alt Space opens the system window menu. This is a Windows shortcut, not a browser-only restriction; it can also intercept the chord in native terminal use. In our browser version, Ctrl Space is the alternate chord for showing the current profile. This alias is not currently provided by our native terminal keymaps.

Browser: Alt Ctrl PgUp / Alt Ctrl PgDn are fixed aliases for Ctrl PgUp / Ctrl PgDn when the browser captures those chords. They use the selected profile's existing application navigation bindings, without changes to Turbo or FAND. The browser aliases are available on all browser hosts.

Their routing is:

- Ctrl Space shows the current profile, like Alt Space. It takes precedence over guest Ctrl+Space input in the browser.
- Alt Ctrl PgUp / Alt Ctrl PgDn deliver logical Ctrl PgUp / Ctrl PgDn through the selected profile's existing bindings. Turbo and FAND need no installer changes. Use Left Alt; AltGr remains reserved for layout input.

These are browser aliases only. Ctrl Shift PgUp / Ctrl Shift PgDn still control zoom, and adding Alt to those still controls backlight. The host must deliver the alias too; custom OS shortcuts can intercept it.

Find details by task

Task	Reference sections
Boot/configuration	Global settings; INI diagnostics; display geometry; build guide for a fresh host.
Application keys	Installed application controls: Turbo/TINST, MBASIC, BBC BASIC, Czech positions and FAND status.
Profiles/history	Global terminal keys; histories/banks; application profiles; editing details; bank settings.
7XF/files	CP/M commands and user areas; shared A0 commands; boot/SUBMIT handling.
Colors/backlight	Global terminal keys; global settings; palette slots and ANSI mapping.
Zoom/pan	Physical chord mappings; display and per-bank geometry; VTINFO/VTZOOM.
Modes/graphics	Output interpretations; escape-sequence subset; xterm256 graphics; visual tests.
Transfer	VTPUT/VTGET from the mirror; ZIP transfer/extraction.

1. Terminal operation

Starting the PC mirror

For hardware on Linux, launch `mirror.sh` in the selected board folder:

```
cd "src/RunCPM_7xtgv-terminal/tools/targets/EP4_10 -- jc8012p4a1c"
./mirror.sh
```

- Choose the folder matching the connected board; the example is EP4_10.
- This attaches the mirror without rebuilding or uploading firmware.
- For desktop on Linux, use `desktop-local.sh` or `desktop-card.sh` under `tools/targets/`. On Windows, use `desktop-local.cmd` or `desktop-card.cmd` under `tools-win/targets/`. Each launches the emulator with its own mirror; the Windows card launcher requires an explicit `--card-root PATH`. Do not attach a hardware mirror to that session.
- See the Build Guide for setup, all board folders and launcher options.

The lower-level `tools/kon.sh` and `tools/kit.sh` launch the hardware mirror in Konsole or Kitty. Both attach to the shared 7xtgv-terminal hardware mirror. Its tmux session (`7xtgv-terminal`) owns the USB serial device. Do not run a second serial reader beside it.

A USB reconnect can briefly blink the PC screen while a complete terminal snapshot is redrawn. This mirror-only refresh preserves a retained xterm256 image on the TFT; it does not restart CP/M.

Paths such as `tools/terminal.conf` below are relative to `src/RunCPM_7xtgv-terminal/`, the sketch directory. The launchers read that shell configuration file; it is separate from the device INI. `XTGV_TARGET` selects the hardware target, `XTGV_PORT` its serial node, and `XTGV_TRANSFER_ROOT` the local transfer-folder root. No second serial reader should run during a transfer.

Ctrl] then q leaves the host bridge. Ctrl] followed by Ctrl] sends a literal Ctrl] byte to CP/M.

Host setting	Effect
`XTGV_TARGET`	Exact named target selected in `tools/terminal.conf`; not inferred from USB identity.
`XTGV_PORT`	Optional serial-port override; absent means target JSON default/discovery. Explicit CLI `--port` wins over it.
`XTGV_TRANSFER_ROOT`	Optional local root for VTPUT/VTGET; defaults to `XTGV_CARD_ROOT` or this repository's `cpmcard`.
`XTGV_REFLASH_COMMAND`	Optional host dispatcher for a guest REFLASH request. Leaving it unset disables automatic flashing.

Host target metadata lives in `tools/targets/<target>.json`; the device INI does not select an uploader target. Restart the mirror after changing host target or port configuration.

Global terminal keys

Physical chords to internal controls

`CSI` means ESC followed by `[`. These controls are not CP/M command strings. Supplied host keymaps deliver them; desktop shortcuts may intercept them. Alt Q invokes configured application exit in Konsole, Kitty and Windows Mintty. Konsole/Kitty retain Alt Esc as an alias; Windows reserves it for window cycling. The browser encoder also accepts both, subject to host delivery; the MacBook observation above found only Alt Esc working. Both encode `CSI >60v`. Pause then x is a native prefix alternative, not a browser mapping.

Physical chord	Internal control / route	Meaning
Alt PgUp / Alt PgDn	CSI >12v / >13v	Previous / next profile, with immediate label redraw.
Alt 0 through Alt 9	CSI >20v through >29v	Select profile 0 through 9.
Alt Space	CSI >14v	Show selected profile.
Alt #	CSI >15v	Toggle RAW input (CZ: Alt AltGr X; US: Alt Shift 3).
Alt Backspace / Alt Q	CSI >61v / >60v	Configured clear / exit action.
Alt Up / Alt Down	CSI >38v / >39v	Profile key events, normally alternate history.
Up / Down	Profile key events	History or app navigation, according to profile.
Ctrl Shift Backspace	CSI >65v	Delete recalled history entry.
Ctrl Shift PgDn / Ctrl Shift PgUp	CSI >66v / >67v	Zoom in (toward 4x) / out (toward 1x) without resizing logical screen.
Ctrl Shift with an arrow key	Versioned CSI >70 key events	Pan by axis, not ordinary guest arrows.
Ctrl Shift Home / Ctrl Shift End	Versioned CSI >70 key events	Scan viewports backward / forward, left to right then top to bottom.
Alt Shift Home / Alt Shift End	CSI 1;3H / 1;3F	Legacy horizontal-only pan aliases.
Alt Ctrl Shift PgUp / Alt Ctrl Shift PgDn	CSI >63;33v / >63;34v	Backlight up / down one level.
Alt Ctrl Shift 0	CSI >63;0v	Backlight OFF.
Ctrl Shift 1 through Ctrl Shift 9	CSI >63;Nv; N=4,7,11,14,18,21,25,28,31	Still-supported coarse brightness.

Physical chord	Internal control / route	Meaning
Alt Ctrl Shift 1 / Alt Ctrl Shift 2	CSI >1v / >2v	ANSI/VT100 / VT52 interpretation.
Alt Shift Backspace	CSI >4v	Native supplied keymaps: arm GCLS graphics without clearing the screen.
Ctrl Alt C	CSI >70;1;52;5;v	Emergency COM return, not guest Ctrl C.
Ctrl Shift X	Native host bridge interception	Native transfer panel; unavailable in the standalone browser.

Exact generated key-event payloads/modifiers are maintained in `tools/7xtgv-terminal.keytab` and `tools/kitty-cpm.conf`; see also the protocol reference. Intended Alt use does not imply Konsole can distinguish both Alt keys.

Profile digits and reserved controls

- Alt 0 through Alt 9 work with both the main row and numeric keypad. The host bridge also recognizes Czech `+ěščřžýáíé` as those row positions. Selection briefly displays the `#n name` label.
- Alt Q and Alt Backspace invoke the global exit/clear policy, configured as `exit_key=escape_alt` and `clear_key=backspace_alt`. Use Left Alt; Right Alt (AltGr) belongs to keyboard-layout character entry, not a required second Alt.
- These controls and the Ctrl Shift view controls precede profile bindings and are not saved in history.
- Backlight and palette selections persist. Ordinary zoom/pan neither changes the INI nor resizes the logical terminal.

Emergency return
Ctrl Alt C

- Forces a warm return from an external COM, including in RAW input mode. It does not send guest Ctrl C.

- Cancels remaining `$$$.SUB` work and ends plotting without forcing a full-screen clear. Unsaved work is lost; open files may be incomplete.
- Does not wait for an application keyboard read, but waits for an active host SD operation. A stopped MCU, terminal task or SD driver needs reset.
- Konsole cannot distinguish Alt from AltGr here. If AltGr C produces Ctrl Alt C on your layout, it is unsafe during a COM. Verify the chord on your host/target before relying on it.

ASCII prefix fallbacks

These are **native mirror** prefix commands. In the standalone browser, Ctrl] is ordinary guest input; none of these sequences opens a native command. Click or tap a page control to leave display input. The commands below are intercepted by the host bridge or terminal before ordinary profile bindings. They do not become CP/M keystrokes; some deliberately invoke a profile’s configured command.

Key	Action
Ctrl] then 1	Select ANSI/VT100 interpretation.
Ctrl] then 2	Select VT52 interpretation.
Ctrl] then r	Repaint normal terminal text on the TFT and PC. This also ends an active xterm256 plot.
Ctrl] then g	GCLS: arm the direct xterm256 graphics canvas below terminal cells.
Ctrl] then a / Ctrl] then e	Move to the previous / next calculated horizontal viewport position.
Ctrl] then p / Ctrl] then n	Previous / next history entry of the active effective bank. Text before the cursor filters entry prefixes, including spaces; a word fallback supports inline completion.
Ctrl] then d	Delete the currently recalled history entry, when one is selected.
Ctrl] then + / Ctrl] then -	Increase / decrease glyph zoom.
Ctrl] then [/ Ctrl] then]	Select preceding / next application profile.
Ctrl] then #	Toggle `## RAW` direct-input mode.
Ctrl] then x	Send the selected profile’s configured exit command.
Ctrl] then c	Send the selected profile’s configured clear command.
Ctrl] then m	Open the host-only VTPUT/VTGET transfer panel.
Ctrl] then b, then 0 through 9	Backlight off, then coarse module-specific levels up to full ON.
Ctrl] then b, then + / -	Raise / lower one of 32 backlight levels.

2. Applications and input

Application profiles

Each of the ten slots starts with the same neutral profile: a `Bank N` label, inactive navigation bindings, no application commands or input adapter, no prompt prefixes and no history-clear sequence or retype delay. A slot number never selects MBASIC, BBC BASIC, TE, Turbo or COMAL behavior.

Application names, key bindings, commands and editing settings belong in their complete `[bankN]` section in `A0:7XTGVTRM.INI`. The working card currently assigns 0 CP/M, 1 MBASIC, 2 BBC BASIC, 3 COMAL, 4 Turbo Pascal, 5 FAND, 6 DDT, 7 TE, 8 ANSI APP and 9 GENERAL. These are card settings, not compiled assignments.

Move the whole section and change only its heading when renumbering. Define the application's required keys (or an explicit `input` preset), commands and editing behavior in that section; omitted settings keep the same neutral fallback in every slot. A `name` or `on_select` value does not select an application adapter. There is no bank-to-bank inheritance or application-profile lookup by name.

`[global] ccp_bank=0` explicitly assigns the CP/M shell profile on this card. If moving that profile, update `ccp_bank` to its new slot as well. Omitted `ccp_bank`, or `ccp_bank=none`, disables automatic CCP routing and native CCP recall handling; it does not prevent ordinary typing at the CP/M prompt.

INI overrides and selection commands

- Every bank is configurable. Empty application commands remain inactive; define `on_select` explicitly when a profile should change the CP/M area.
- `on_select` is emitted only when switching at a live CCP prompt, never inside TE, Turbo or another application. For example, `cd /b/mbasic\r` selects `mbasic(B1)` because CCP executes it; bank numbers do not identify DUs.
- The temporary label restores its saved row and cursor without requiring an application redraw. Wait for it to disappear before normal input.
- Alt Space only shows the label; it never emits `on_select`.

Histories, banks, and the active line

There are ten named FIFO histories, each retaining at most 100 entered lines. Single-character submissions are ignored.

Each entry retains the first 79 text characters: its saved `>` marker makes an 80-character record, excluding the line ending. This limit applies to every bank and to loading older, longer entries. Active-line editing still tracks up to 127 characters; only the retained history is shortened. A later history save writes the shortened entries, so archive an older BUF separately if its longer entries are needed.

- Up/Down normally recall entries; profiles may reserve them for the app and use Alt Up / Alt Down for history.
- Recall first matches the entire text before the cursor against history-entry prefixes, ignoring ASCII letter case and retaining spaces and parameters: `cd /` selects entries starting with `cd /`, and `cd ` includes the space. If no full-prefix match exists, a nonempty word immediately before the cursor is tried instead, preserving inline completion such as `10 pr` from `PRINT`. A query ending in whitespace stays a full-prefix query; no match leaves the line intact. At column zero, recall browses without a filter.
- Filtered recall wraps within the chosen matches and replaces the matched prefix or word, preserving the surrounding text and suffix after the cursor. It inserts the retained entry exactly, subject to the line-length limit: no space is appended or inserted. Spaces already in the entry or surrounding text are preserved.
- Enter stores the submitted line, including an unchanged filtered recall or a completion within a prefix such as `10 PR`. Merely browsing or cancelling a recall does not add an entry.
- An exact repeat of the newest entry in that bank is not appended again. This comparison is byte-for-byte, including spaces; it is not global deduplication. A recalled older command becomes recent again on submission.
- Spaces/tabs are literal in a full-prefix filter and delimit the word fallback; prefer spaces in authored CP/M text.
- Ctrl Shift Backspace deletes the selected recalled entry.

- Separate editable keyword lists remain planned, not implemented.

Persistence

- Histories live in `A0:7XTGVTRM.BUF`; saves follow a quiet period or `SYNC`.
- Boot can recover from verified alternate generations.
- Do not edit checksummed history files as text. Preserve backups and use checksum-aware tooling for repairs.

Storage transactions and core ownership are described in the Technical Reference Guide.

Selected profile versus effective history bank

- Profile selection lasts for the running session; it is not persisted in the INI.
- A recognized RunCPM CCP prompt (`A0>`, `AF>`, `A>`, and similar) silently uses effective bank `#0` so shell commands stay out of application history.
- The selected bank is used again when the application prompt returns.
- Active-line and cursor colours follow the palette unless explicitly overridden by the profile. PAPER's pale-green/mid-green defaults are not fixed across themes.

Editing details

The normal terminal is a line model, not a generic editor. It records a line when Enter completes it and can recall it into line-oriented interpreters.

MBASIC
two adapter choices

- `EDIT n` enters MBASIC's modal editor; it needs application-specific handling.
- Select `input_adapter=mbasic_native` explicitly for native EDIT operations: rubout and Space movement, L, X then Esc, and 1 then D.
- The working MBASIC profile selects `local_compose`: local Left / Right, Home / End, Insert, Delete, Backspace and line submission.

- The composer’s line-buffer machinery could be reused, but its current adapter also recognizes `EDIT n` and bridges MBASIC’s modal editor: issue X, capture the displayed numbered line, then Esc and Q before local editing. It assumes compatible Backspace/echo/submission behavior; other applications are not qualified by that implementation. It is not a full-screen editor.
- Adapter-owned keys are consumed before their configured bytes are sent; see loading order and key priority. Changing `input` does not disable an adapter. Use `input_adapter=none` for TE/FAND/Turbo byte mappings; keep MBASIC’s chosen adapter paired with MBASIC.

BBC BASIC
visible prompt and retyping

- Its visible `>` prompt identifies the active screen line.
- Ordinary edits are tracked; recalled history uses the configured clear/retype sequence. Its controls therefore differ from MBASIC’s.

Our installed application controls

This is the working-card keyboard reference, reviewed on 261014. It describes our experiments and current configuration, not the original vendors' defaults. Select the matching application profile before launching a program. A physical card, an older browser ZIP or another copy of an executable can contain a different installation.

Package / profile	Current approach
`mbasic(B1)` / bank 1	Local line composition and an MBASIC EDIT bridge; see adapter details and the quick keys below.
`bbcbasic(B2)` / bank 2	BBC's own line editor, explicit control bytes and terminal history; also used with `BBCLOWER`.
`comal(B3)` / bank 3	Reserved package area and an unverified ANSI input template. No COMAL runtime is installed here.
`tp(B4)` / bank 4	Our TINST-installed Turbo Pascal controls, including private single-byte keys. A matching `TURBO.COM` is also installed in `cpm(A0)`.
`fand(B5)` / bank 5	Experimental WordStar-family input mappings and console adaptation. Screen behaviour and controls remain incompletely working.
`dbase(B6)`	Imported package/data; this guide does not establish a tested dBASE keyboard installation. Bank 6 is currently DDT, not dBASE.

How the TINST and FANDINST experiments developed

We were investigating how real CP/M applications receive keys and drive the screen, while making Turbo comfortable on our Czech keyboard. Familiar PC navigation and memorable editing chords were the starting point; reproducing a traditional WordStar keyboard was not the objective. The result is a practical, partly experimental installation rather than a universal keymap.

Four separate parts have to agree:

```
physical key + modifiers
  -> terminal profile: bytes sent to CP/M
  -> application's installed input commands: editor action
  -> application's output definition: bytes drawn by the terminal
```

Early Turbo experiments used ANSI arrow strings and Ctrl-letter prefixes. Some surprising results came from the application's interpretation of those prefixes: the old Ctrl+L, then i insertion experiment made Turbo emit an insert-line command while its cursor was on the status row. The same effect occurred in the native CP/M comparison; changing the terminal's normal ANSI insert-line semantics would have hidden the wrong problem.

TINST then proved able to accept bytes with bit 7 set. This allowed compact, unambiguous single-byte controls for the main navigation/editing keys. An important installation discovery was that **TINST installs editor commands into the selected** `TURBO.COM`. Its `TINST.DTA` terminal-definition library is not a substitute for the installed executable. Configuring one copy and launching another explained an apparent failure of the high-bit keys. The subsequent device test confirmed the private-byte approach in Turbo.

FANDINST was a related investigation into the same separation of input and output definitions. Turbo's success did not prove FAND's matcher, patch target, secondary controls or console protocol. FAND remains an unresolved adaptation; see the FAND status below.

Historical evidence: the early Turbo experiments and private-byte proof. Their earlier mappings and card paths are historical, not the current tables below.

Czech keyboard positions QWERTZ and QWERTY

The following original annotated photograph records the position names used in our INI. They identify **physical positions**, not the character currently printed by a Czech, US or other host layout.

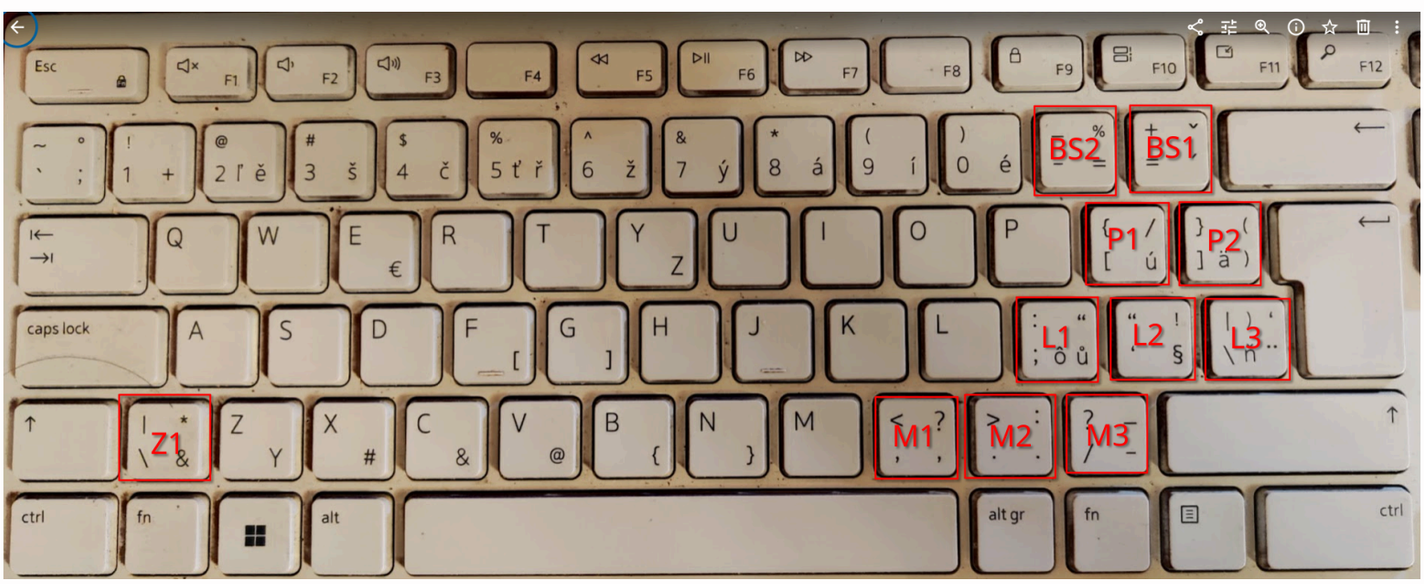


Figure 1. Czech keyboard position names from the original investigation

INI position	Where it is
`num1` through `num0`	The main number row, separate from the keypad. On the observed Czech layout its unshifted legends are `+ěščřžýáié`.
`bs2`, `bs1`	The two positions immediately before **Backspace**, left to right.
`p1`, `p2`	The two positions immediately after **P**.
`l1`, `l2`, `l3`	The three positions immediately after **L**, before **Enter**.
`z1`	The extra ISO key between left **Shift** and the first bottom-row letter: **Y** in QWERTZ, **Z** in QWERTY.
`m1`, `m2`, `m3`	The three positions immediately after **M**.

QWERTZ and QWERTY exchange the Y/Z letter positions; `z1` keeps its existing INI name in either layout. The photograph carries several legends per key and is not a promise that every host layout produces those same characters. The observed Czech `bs1` and `l3` positions are dead accents: ordinary typing must retain composition. **AltGr** is for layout symbols, not the `Alt` bit in the private encoding below. See key-binding syntax for the supported names and host limitations.

The compact one-byte scheme

The original marked keyboard assigns a low hexadecimal nibble to a core key. Its red F label over X was an exploratory choice; it is not a current instruction to encode the letter X as a special key.



Figure 2. Original core-key nibble sketch; slot F was not settled

```
bit       7   6   5   4   3 2 1 0
meaning   1  Ctrl Alt Shift key number

byte = 0x80 | Ctrl*0x40 | Alt*0x20 | Shift*0x10 | key
```

High nibble	Modifiers
`8`	None
`9`	Shift
`A`	Alt
`B`	Alt+Shift
`C`	Ctrl
`D`	Ctrl+Shift
`E`	Ctrl+Alt
`F`	Ctrl+Alt+Shift

Low nibble	Core key
`0`, `1`, `2`, `3`	Up, Left, Down, Right
`4`, `5`, `6`, `7`	PgDn, PgUp, End, Home
`8`, `9`	Delete, Insert
`A`, `B`	Tab, Space
`C`, `D`, `E`	Esc, Backspace, Enter
`F`	Unsettled physical identity / extension design; see the limitations below.

Thus Left is `81`, Ctrl Left is `C1`, and Shift Delete is `98`. Hex values here are raw input bytes, not printable hexadecimal text. For example, `left_ctrl=send:\xC1` emits one byte.

This is an application-facing convention implemented by explicit profile bindings. It is **not** automatically applied to every special key or every application, and is separate from the host's `CSI >70` key-event transport. Terminal-owned chords still take precedence. Plain Tab, Backspace and Enter, for example, do not become `8A`, `8D` and `8E` merely because the design table lists them.

The working Turbo INI experiments with `z1_ctrl=send:\xCF` and `z1_ctrl_shift=send:\xDF`; neither byte appears in the current 45-entry installed primary command table. The proposed `8F` extension prefix has no implemented extension grammar. `FF` is TINST's Nothing value in the inspected installation, not an available Turbo command simply because it fits the formula.

Turbo Pascal installed controls

The following tables combine the current bank-4 INI with the installed 45-entry command table read from both `cpm(A0)` and `tp(B4)` copies of `TURBO.COM` on 261014. Those executable copies match. This records the stored configuration, not a fresh interactive proof of every action on every host. TINST's action names are retained: for example, Scroll down and Scroll up refer to its named commands, not a redefinition of their visual effect.

Table 1. Hexadecimal bytes shown below are the configured guest input

Physical key	Hex	Installed Turbo action
Left / Right	81 / 83	Character left / right
Up / Down	80 / 82	Line up / down
Ctrl Left / Ctrl Right	C1 / C3	Word left / right
Ctrl Up / Ctrl Down	C0 / C2	Scroll down / up
PgUp / PgDn	85 / 84	Page up / down
Home / End	87 / 86	Left / right on line
Ctrl PgUp / Ctrl PgDn	C5 / C4	Top / bottom of page
Ctrl Home / Ctrl End	C7 / C6	Top / end of file
Alt Home / Alt End	A7 / A6	Beginning / end of block
Insert	89	Insert mode on/off
Ctrl Insert	C9	Insert line
Delete	88	Delete character under cursor
Backspace	08	Delete left character
Ctrl Delete	C8	Delete line
Shift Delete	98	Delete to end of line
Esc	0B 04	End edit, via the profile’s Ctrl+K Ctrl+D sequence

Some remaining installed commands still use ordinary control bytes or multi-byte sequences. In the next table, `^X` means the Ctrl-X byte `18`, and a following lowercase letter means that literal letter. These are byte notations, not additional private-nibble assignments. Producing NUL or Ctrl-backslash on a Czech layout is host-dependent; no universal physical chord is asserted for those entries.

Installed action	Input notation	Hex
To last cursor position	NUL	`00`
Delete right word	`^D`	`04`
Mark block begin	Ctrl-backslash byte	`1C`
Mark block end	`^Y`	`19`
Mark single word	`^W`	`17`
Hide/display block	`^X ^X`	`18 18`
Copy block	`^X c`	`18 63`
Move block	`^X v`	`18 76`
Delete block	`^X`, then the private Delete byte	`18 88`
Read block from disk	`^X r`	`18 72`
Write block to disk	`^X w`	`18 77`
End edit	`^K ^D`	`0B 04`
Tab	`^I`	`09`
Auto-tab on/off	`^T`	`14`
Restore line	`^L r`	`0C 72`
Find	`^Q ^F`	`11 06`
Find and replace	`^Q ^R`	`11 12`
Repeat last find	`^Q ^Q`	`11 11`
Control-character prefix	`^C`	`03`
Character-left alternative; delete-left alternative	Nothing	`FF` in each slot

Additional working-profile details:

- Ctrl S is an experimental quick-save macro: `^K ^D S E ^Q ^P`. Its intended flow is end edit, save, reopen the editor and return to the previous position. It assumes the expected Turbo menu/editor state; it is not an unconditional save operation in every dialogue.

- Alt Q sends the configured `Q`, appropriate to Turbo's menu Quit. It is not the same as the editor's End edit command. Use Esc to leave the editor first; there is no promise that `Q` exits every Turbo context.
- Alt Up / Alt Down remain terminal history. The INI sends ANSI fallbacks for Alt Left / Alt Right, not the proposed `A1` / `A3`.
- The INI emits further Shift/Alt private codes not assigned in the primary table. An emitted byte does not by itself establish an editor action; Turbo's secondary controls and host-reserved chords need separate checks.
- TINST's output definition is independent. For example, terminal reset on leaving Turbo should include `ESC [ 0 m` to clear emphasis; initialization and exit reset are different fields.

The installer capability list is in the TINST reference. The reproducible binary readout and file identity for this review are recorded in the documentation checkpoint.

MBASIC and BBC BASIC quick keys

MBASIC's current `local_compose` adapter consumes its editing keys before ordinary profile-byte mappings. BBC BASIC uses its own editor controls from `DIST.Z80`. Their differences are intentional; selecting a shared ANSI or WordStar input preset does not install either interpreter's editor.

Key / operation	MBASIC, bank 1	BBC BASIC / BBCLOWER, bank 2
Left / Right	Move in the local composition line.	Send `0A` / `0C`, BBC cursor left / right.
Home / End	Beginning / end of the local line.	Send `06` / `0E`, start / end of line.
Insert	Toggle local insert/overwrite.	Send `11`, BBC insert/overwrite toggle.
Delete / Backspace	Delete at / before the local cursor.	Send `12` / `08`, delete at / before cursor.
Up / Down	Terminal history previous / next.	Terminal history previous / next.
Alt Up / Alt Down	Explicitly disabled in this profile.	Send `07` / `0F`, BBC's native cursor up / down controls.
Recall preparation	Local composition handles the line.	Send `06 18` to start of line and delete its remainder; wait the configured 100 ms before retyping.
Alt Backspace at the prompt	Send the configured PRINT expression to clear/home.	Send the configured PRINT expression to clear/home.
Alt Q at the prompt	Send `system` and Enter.	Send `quit` and Enter.

In MBASIC, `EDIT n` also uses the adapter's interpreter-specific bridge, described in Editing details. Without that adapter, the INI's raw `L`, `X`/Esc and `1D` editing sequences have different, modal meanings; they are not general cursor controls.

The BBC profile sends ANSI sequences for PgUp/PgDn and additional modifiers; this does not establish that BBC's editor implements those actions. Its core byte meanings above are documented in the installed `bbcbasic(B2)` `DIST.Z80` source. `BBCLOWER` changes keyword recognition, not this key table.

Application exit/clear commands are ordinary guest input, not forced process termination. At an interpreter prompt they execute its commands. A running program must understand the input itself: our MBASIC VTPAL/VTESTS handle the complete SYSTEM sequence and return to OK, while arbitrary BASIC programs do not inherit that behaviour. See command timing.

FANDINST, the Sharp preset and unresolved output

FAND's keyboard and display remain experimental. The current bank-5 bindings use WordStar-family input: arrows send `^S`, `^D`, `^E`, `^X`; Ctrl+Left/Right send `^A`/`^F`; Home/End send `^Q ^S`/`^Q ^D`; PgUp/PgDn send `^R`/`^C`. There are additional Ctrl/Shift commands and a `^K ^S` save macro on Ctrl+S. These are the bytes we configure, not a claim that every action works correctly in every FAND editor or dialogue. They do not use Turbo's private-byte scheme.

The original FANDINST preset was labelled **SHARP MZ-821**, and the application identified itself as `FAND-SHARP MZ 821 & K 6314(13)`. That establishes a Sharp-targeted installation; it does not identify every internal patch or prove which CP/M BIOS it expected. The archived installer observations include:

Original preset item	Recorded setting
Geometry	80 columns by 25 rows
Clear screen	`0C`, with cursor home after clear
Erase to end of line	`16`
Cursor addressing	`1B`, then binary row and column with offsets of 128 each; row first
Alternate brightness	`1C 33` on; `1C 32` off, shown in zero-padded installer fields

Those controls are not ordinary ANSI or VT52. Changing only `input=wordstar` or matching navigation keys cannot correct an incompatible output protocol. Likewise, changing output interpretation cannot fix wrong input bindings or an installer applied to a different executable/chain file.

The historical investigation considered Lamač / LEC CP/M a plausible console lineage. It did **not** establish that identification. The examined `BIOS8MZ.MAC` for CP/M V2.26-31 disagrees with the FAND preset's clear/EOL controls, so it must not be copied as a supposedly universal MZ-821 definition. A dedicated output personality may eventually be useful if the actual emitted byte stream requires it, but **no Sharp/MZ-821 output mode is implemented or established as necessary by this documentation**. ANSI adaptation remains an experiment, not a completed FAND compatibility claim.

The next useful investigation is to identify exactly which executable/chain files FANDINST changes and capture the failing application's actual output, then compare that with the stored screen definition. This separates an installation mismatch from a decoder gap before selecting a custom mode. See the public Sharp/FAND investigation and BIOS comparison.

RAW input

- `## RAW` bypasses normal profile/history translation for direct application input.
- Use it for diagnostic tools or unusual full-screen programs; return to normal input for ordinary command work.
- Direct Alt 0 through Alt 9 selection leaves RAW mode; previous/next profile browsing does not. The RAW toggle can switch it off without selecting another bank.

3. CP/M commands and files

7XF shell and user areas

The original RunCPM `VOL [drive]` command reads the first printable line of `INFO.TXT` in each existing user area on that drive. It does not read the `.7XF` catalog files used by the extensions below.

This compact reference covers files added or adapted by this project. It does not repeat ordinary CP/M programs and imported language packages; use `?` at the CCP prompt for the internal CCP command list. An A0 command is found from other user areas by this RunCPM CCP, while files it creates normally belong to the current DU (for example `C3>`).

7XF prompt style

`7XF` is an internal CCP command; there is no `7XF.COM` to install. The selected style lasts until RunCPM restarts. `LS`, `LL` and `CD` work without running `7XF`; it changes prompt presentation, not command availability. There is no dedicated prompt-style INI setting. To select named prompts on boot, put the single command `7XF` in the card-root `AUTOEXEC.TXT` (not inside A0). See Boot, AUTOEXEC, and pending SUBMIT work.

Command	Prompt
`7XF DD`	Decimal user number, e.g. `A15>`; boot default.
`7XF DH`	Hexadecimal user number, e.g. `AF>`.
`7XF D`	Drive only, e.g. `A>`.
`7XF` or `7XF DN`	Named area, e.g. `/b/mbasic>`; `/b/1>` without an alias. Names are at most eight characters.

Listing and selecting areas

Slash paths are names for **flat CP/M drive/user areas**, not subdirectories. Lookup is case-insensitive; displayed names retain their catalog spelling. `/` is only a drive overview, not a selectable root directory. `CD /B` selects the remembered B-drive area; `CD /B/MBASIC` selects that named DU explicitly.

Command	Result
`LS`	Short listing of the current DU.
`LL` or `LS -L`	Long listing, including sizes and descriptions.
`LS /` or `LL /`	Nonempty drives, totals and descriptions; LL adds record/DEXT counts.
`LS ..` or `LL ..`	Populated user areas on the current drive, equivalent to `/A` when on A, `/C` when on C, etc. Does not change DU; empty/catalog-only areas are omitted.
`LS /A`	Drive A description, then populated areas and their totals.
`LS /B/MBASIC/.BAS` **or** `LS B1: .BAS`	Files matching CP/M `*` / `?` wildcards in `mbasic(B1)`.
`LS -O PANE.7XF /B/MBASIC`	Replace `PANE.7XF` in the current DU with that listing; no terminal listing or PAGE pause for this invocation.
`PAGE 0`	Disable ordinary pauses in LS, LL and `?`.
`CD`	Report numeric DU and named view, e.g. `B1: /b/mbasic`.
`CD MBASIC` or `CD /B/MBASIC`	Select a named area.
`CD /A`	Return to the last selected area on drive A.

- With no dot, a stem **spans the extension too: `LS *` lists every file; `LS VT`** includes `VTESTS.BAS` and `VTPAL.BAS`.
- This also applies after an area path or `B1:`. An explicit dot keeps normal 8.3-field matching.
- Files always sort by extension, then filename (case-insensitive); names still display as lowercase `name.ext`, without renaming host files. Extensionless files come first. This also applies to filtered/path listings and `-O` output. Drives/areas retain drive letter/user-number order.
- All listing rows clip one column short of the logical terminal width to avoid wrapping; a wider logical screen shows more description text.

- `LS -O` captures only that listing, not CD, other built-ins or COM output.
- Area overviews (`LS /A`, `LL /A`, or `..` for the current drive) omit empty areas and those containing only `DIR.7XF` and hidden host placeholders. A zero-byte ordinary file counts as content. Omitted areas remain accessible by DU or catalog alias; direct file listings still show their catalog.

Listing sizes and descriptions

- Short rows show comma-grouped KiB (`k`), rounded up. File descriptions are omitted; the current-area heading retains its description. Root drive overviews retain drive descriptions in both forms. In area overviews, LS omits individual area descriptions; LL includes them. A drive's area list starts with that drive's `A0:VOL.7XF` description at column 2, in both `LS /drive` and `LL /drive` (also their `..` aliases).
- Long rows show exact host-file bytes, 128-byte record counts, rounded-up 32-KiB DEXT equivalents and per-file descriptions.
- Drive/area totals sum these values per file. DEXT is an estimate, not an actual CP/M directory-entry count; none of these is FAT-cluster allocation.
- Descriptions stay on the same row: file/path column, two-space separator, then description. When listing the current area, its heading contains only the description, starting at column 2; the prompt already gives its path. A different area's heading retains its path and description. Width affects clipping, not the choice of a one-line/two-line layout.
- CP/M record writes may already have rounded a host file to 128 bytes. Its host-file length is not necessarily the original payload length; text `Ctrl-Z` is not a reliable binary end marker.

Catalog files

File	Format and use
Each DU's `DIR.7XF`	12-character left-aligned area/file key, one space, description. The first key names the area.
`A0:VOL.7XF`	Drive letter in column 1, space in column 2, description from column 3.
`A0:DRV.7XF`	Last selected area per drive, one `/d/name` line per drive. CD reloads it before selection and saves after a new explicit selection.

The working card keeps these files as CRLF text ending with `Ctrl-Z`. They do not replace the original VOL command's `INFO.TXT` input.

Project files by DU

Name(DU)	Contents
`cpm(A0)`	Shared installed helpers listed below, ordinary CP/M tools, INI and history.
`cpmdev(A1)`	Historical CP/M/CCP sources, assemblers and their required libraries.
`mbasic(B1)`	Full Microsoft BASIC package, variants and general examples.
`bbcbasic(B2)`	BBC BASIC package including project-built `BBCLOWER`.
`comal(B3)`	Reserved catalog/placeholder files; no COMAL runtime or demo installed yet.
`tp(B4)`	Complete Turbo Pascal package and examples.
`7xtgv(C1)`	Canonical guest helper sources/includes, HEX/listings and visual BASIC tests; MBASIC included.
`xterm256(C2)`	Brot scripts/sources, graphics/timing helper COM files and attribution note.
`bench(C3)`	CPU benchmarks/exercisers and VTDRV disk fixture.
`fand(B5)` / `dbase(B6)`	Imported database packages and local data; not project commands.

B0 and C0 intentionally contain no shared commands. See the card README for source/build ownership and package boundaries. The physical card is a separate copy, not automatically reorganized with the repository.

Shared A0 commands

Command	Purpose
`REBOOT`	Requests a module restart; the hardware bridge attempts to reconnect after USB returns. On desktop the launcher starts a fresh emulator process. Relaunch the appropriate mirror only if it does not recover.
`REFLASH`	First forces and verifies the same history save as `SYNC`, then emits the reserved host event `ESC[>64v`. It flashes nothing by itself; an attached `terminal.py` acts only when its target-specific `XTGV_REFLASH_COMMAND` is explicitly configured.
`ZOOM [1|2|3|4]`	Temporarily set 1x..4x glyph zoom without resizing CP/M; no argument means 1x. Source: `7xtgv(C1)` `ZOOM.ASM`.
`VTINFO`	Report display ID, exact firmware target/build, physical panel, current logical/visible view and resolved presets. Requires firmware with `CSI >72v` reply support.
`VTZOOM`, `VTZOOM n`, `VTZOOM n COLSxROWS`, `VTZOOM DEFAULT`	Inspect the view; temporarily load preset `n`; select explicit logical geometry; or restore the INI/HAL boot view. No form saves the INI. A full-screen application's installed dimensions should match before it starts.

Command	Purpose
`VTPAL [0..9]`	Select an editable text/UI palette; no argument selects 0. Uses private output `CSI >69;n v`, with lazy INI persistence. See the palette manual. Requires the new firmware and helper.
`UNSUB`	Deletes a stranded `$$$.SUB` queue in the current DU without switching drive or user. It is silent if none exists.
`SPEED n`	Set RunCPM CPU throttling, `0..65535`. `0` disables throttling; `65535` is the fastest throttled setting. See SPEED details.
`VTCPU` / `VTCPU8X`	Measure the RunCPM interpreter with the terminal active and paused. `8X` uses the 8080/8085 common instruction subset.
`VTDRV`	Plain invocation read-tests and verifies a retained 32 KiB `VTDRV.$$$` fixture. `VTDRV W` recreates it, then reads/verifies; old `RW` remains an alias. `VTDRV Q` quietly reads without writing. `VTDRV W Q` (or `Q W`) pauses the terminal during both timed write and read passes. Quiet modes require matching firmware; options are case-insensitive whole tokens.
`VTDRV8X`	Write, read, and verify 32 KiB in the current DU, then remove its temporary test file. Uses the 8080/8085 instruction subset.

Command	Purpose
`CARDMARK`	Native raw SD read diagnostic on direct-SdFat/SPI targets, launched by A0:CARDMARK.COM. Prints CID/CSD identity and compares repeated raw sector reads; see below. Does not write a test file.
`VTTERM`	Run the native terminal/parser and display redraw benchmark; report decimal kChars/s and kB/s (M at one million units/s). See measurement details.
`VTGET` / `VTPUT`	CP/M ends of the host `tools/cpm-transfer/transfer.py` protocol; normally use the host tool rather than type their protocol manually.
`UNZIP157 FILE.ZIP A:`	Extract a Linux ZIP/Deflate archive to drive A in the current user area. Omitting the destination lists/checks only. Installed separately from old `UNZIP.COM`.

`UNSUB.ASM` is retained in `7xtgv(C1)`; rebuild/deploy `UNSUB.COM` after a card replacement if it is absent.

`VTINFO` prints a report like this (illustrative values only; the running firmware and its resolved INI/profile settings provide the actual values):

```
Display : RP2_147
Target  : rp2350-lcd-1-47-a
Panel   : 320x172
Build   : <build-stamp>
View    : zoom 2, logical 80x24, visible 26x8

Resolved presets:
1x: 80x24
2x: 80x24
3x: 26x5
4x: 20x4
```

`VTZOOM` does not notify ordinary CP/M applications of a size change; select matching logical geometry before launching a full-screen application. The `7xtgv(C1)` ASCII sources are `VTINFO.ASM` and `VTZOOM.ASM`; assemble each with `ASM` then `LOAD` if its `.COM` is missing. The private query/reply bytes are in the Technical Reference Guide.

Detailed diagnostic procedures and demonstration recipes are in Appendix A.

SPEED
intentional CPU slowdown

`SPEED n` is an external RunCPM helper installed in `cpm(A0)`, with source `SPEED.Z80` in `7xtgv(C1)`. It accepts a decimal value from 0 through 65535. Without an argument it prints usage; it does not query the current setting.

Command	Meaning
`SPEED 0`	Disable CPU throttling and restore maximum available interpreter throughput. This is the runtime's default.
`SPEED 65535`	The fastest throttled ("slow") setting. It is still distinct from unthrottled operation.
`SPEED n`, with `n` from 1 to 65534	Smaller values impose stronger slowdown; useful when an old program runs too quickly.

The number is an instruction count, not MHz or a percentage. The interpreter checks elapsed time after each group of `n` instructions and waits if that group took less than 10 ms. This is an approximate throttle, not cycle-accurate Z80 timing; it cannot accelerate a host already slower than the requested rate. The setting remains until changed or the emulator runtime restarts; an ordinary return to CCP does not restore the default.

On 261014, a user test on an Intel Core i7-10710U system (reported as 1.1 GHz) found `VTCPU` reporting approximately **44 MHz** with `SPEED 65535`, in both native desktop and web execution. This is that benchmark's equivalent Z80 rate for its instruction mix, not a fixed meaning of 65535 or a measurement of the host's instantaneous clock. Browser, host and workload affect results. Use `SPEED 0` before comparing unthrottled CPU performance.

SPEED changes emulated instruction execution, not MCU clock, terminal refresh settings or the native VTTERM rendering workload. The helper calls RunCPM's BDOS extension 254; it is not a standard CP/M speed-control command.

VTTERM display benchmark

Run `VTTERM` from the CP/M prompt. The shared command lives in `cpm(A0)`; its current source is VTTERM.Z80 in `7xtgv(C1)` (`cd /c/7xtgv`). The short assembly program is a launcher: private port `79h` starts the native test, reports completion and requests the result. Its small size and old source date do not imply an obsolete benchmark. The workload is in `terminal_runtime.cpp`; the result formatting is supplied by the host runtime. Formatting changes require rebuilding that runtime, with no replacement COM.

Labels use NORMAL and ALT ink names, paired with their actual SGR colour. ALT means the configured alternative ink, which need not be brighter. Each item also chooses a random native paper slot 0..3. The corrected labels and random papers change the workload, so compare runs using the same benchmark revision.

The test resets the terminal, clears each frame and places 96 coloured text items per terminal-loop pass, continuing for at least three seconds. Normal render work between passes contributes to elapsed time. This is a combined parser/render-loop benchmark, not isolated panel bandwidth, a Z80 benchmark or USB serial throughput: the fixture feeds the native parser directly. The last pass ends timing before its subsequent render completes.

Illustrative output:

```
VTTERM native: 27.029 kChars in 3086 ms
  = 8.757 kChars/s (20.591 kB/s)
VTTERM: PASS
```

- The total counts printable fixture characters, shown as `kChars` or `MChars` with three decimal places. `ms` is elapsed whole milliseconds, without fractional digits or thousands separators. Rates are still calculated from the original microsecond measurement.

- `kChars/s` counts those characters per second; `kB/s` also counts the fixture's colour, position and reset escape sequences. The per-frame clear and final result-position sequences are outside the byte counter.
- `k` means 1,000, not 1,024. Each rate switches independently to `MChars/s` or `MB/s` at 1,000,000 units/s. Rates retain exactly three decimal places (truncated); commas group thousands in the integer part. Older runtimes print microseconds and plain chars/s and B/s.
- `PASS` means the native workload completed. It does not check pixels or establish visual correctness. Compare identical geometry, zoom, target, mirror/output configuration and runtime build; inspect the display too.

Boot, AUTOEXEC, and pending SUBMIT work

`AUTOEXEC.TXT` is a RunCPM convenience, not a standard CP/M 2.2 CCP command. It is a short first-boot command line at the SD-card filesystem root (outside every A0—AF DU). For deliberate startup, it can contain `SUBMIT STARTUP`, with ordinary `STARTUP.SUB` held in the intended CP/M DU.

`SUBMIT` builds the internal `$$$.SUB` queue. On boot the CCP can find such a pending queue and resume it; that is useful after interruption but is not a safe way to turn ordinary text prefixed with `$` into autorun. If an abandoned queue causes surprising commands after boot, switch to its DU and run `UNSUB`. A pending queue takes precedence over `AUTOEXEC.TXT`.

VTPUT / VTGET from the mirror

This section requires the **native mirror**; the standalone browser uses card ZIP import/export instead.

Leave CP/M idle at its CCP prompt. Open the host panel with Ctrl Shift X (or Ctrl] then m). It invokes the appropriate helper automatically; do not start XMODEM or another application first. An idle-prompt check precedes commands, but this is a screen heuristic, not a firmware state guarantee.

Direction	Choose in panel	Operation
Host file to CP/M	`S`, local DU, source, target, `Y`	VTPUT writes target in device current DU
CP/M file to host	`R`, local DU, source, target	VTGET reads device current DU; asks Y/N/C if local target exists

Local folder and filenames

- Both directions ask for **DU**, **Source** and **Target**. Target initially copies Source and can be edited.
- `c2:` selects local `cpmcard/C/2` (`xterm256(C2)`); `AF:` and `A15:` both select `A/F`. Matching is case-insensitive. Missing or symlinked DU folders are rejected.
- **DU selects only the local folder.** The device remains in its current drive/user. To receive device A0 into local A3, leave CP/M at `A0>` and choose Receive with local DU `A3:`.
- Filenames must be supported 8.3 names, without paths. Lowercase input can match existing uppercase names; new receive names are uppercase.
- `XTGV_TRANSFER_ROOT` in `tools/terminal.conf` selects another local root. The older `XTGV_XMODEM_ROOT` remains a fallback.

Editing and overwrite approval

- Use Left / Right, Home / End, Backspace / Delete to edit; Ctrl U clears the field. Enter submits it. Konsole's encoded editing keys are handled locally.
- Send always asks permission to create/replace the device target; it does not probe remote existence. Receive asks only if the local target exists.
- Y proceeds; N returns to Target; C or Esc cancels. Enter **is not overwrite consent.**
- A replacement is committed only after the complete download and temporary write. A changed destination invalidates earlier approval; new local files use exclusive creation.
- The result remains visible until Enter or Esc.

Bytes and profile recovery

- Bytes, including CP/M record padding, are transferred unchanged. There is no automatic text conversion or compression. CLI `--cpm-text` is for deliberate external host-text imports.
- Transfers temporarily select profile #9 and RAW input. The profile query behind Alt Space captures the original state first.
- Success and failure both restore the original profile and RAW state, without executing `on_select`. Unconfirmed restoration reports an error.
- If CP/M is still running a transfer helper after failure, reset before retrying.

Protocol and logical width

VTGET tags each hex row with its record number to reject stale mirrored rows. The updated host also accepts the old clear-per-record format; old host clients do not understand the new tagged format. The framing implementation is in the Technical Reference Guide.

Use an 80-column **logical terminal** for transfers. At 40 logical columns, VTGET's 69-character tagged rows wrap and the download can stall. A 26- or 40-column physical viewport onto an 80-column logical terminal is different and does not impose that wrapping. RAW input also bypasses normal pan-command interception; do not use interactive pan as a transfer recovery mechanism.

Before changing the device INI, fetch its current A0 copy with `tools/fetch-7xtgvtrm-ini.sh` (explicit `--target`/`--port` if needed), edit the fresh local mirror, then send it with overwrite approval. The helper archives the previous local copy and replaces it only after a successful fetch. Stop on fetch failure: an older local INI can overwrite unrelated on-device edits. See live INI and transfer-width notes.

Guest text in the local `cpmcard/` working tree is maintained as CRLF with a final `Ctrl-Z`, ready for byte-preserving Send/Receive. No upload conversion is needed. The CLI text-conversion option is only for deliberate external host-text imports. Tokenized BASIC, binaries, disk images and checksummed history are not text; host-only JSON metadata, Markdown readmes and Makefiles retain host format.

XMODEM is not supported through this reconstructed-screen channel. Its UI entry was replaced; CONSOLE8 or RAW input alone does not make it binary-safe.

ZIP transfer and extraction

Transfer ZIPs as binary, never with `--cpm-text`. The separately installed `A0:UNZIP157.COM` supports Linux Deflate ZIPs; the old local UNZIP 1.8 does not. Keep uppercase 8.3 member names. Linux packing example:

```
zip -9 -j VTFILES.ZIP FILE.INI
```

After transferring the archive, CP/M extraction is:

```
UNZIP157 VTFILES.ZIP A:
```

The destination enables extraction; without it the program only lists/checks. Use a disposable user area or nonconflicting names for initial tests. Automatic compression in the mirror is not implemented. See the compatibility test and download record.

4. Configuration

INI sections and loading

`A0:7XTGVTRM.INI` is read at boot from the selected card, on hardware and desktop alike. Reload edited definitions with REBOOT. The desktop uses the chosen real board's display ID, not a separate SDL configuration section.

Section	Scope
`[global]`	Output mode, console width in bits, SPI request, palette, brightness, cursor blink, gutter colours and common key policy.
`[display:HAL_ID]`	Boot zoom, per-zoom logical geometry and render-flush interval for the matching display ID.
`[bank0]` through `[bank9]`	Application profile name, commands, key bindings, history/editing behaviour and optional geometry/colour overrides.
`[palette0]` through `[palette9]`	Text inks, papers and UI colours for each selectable palette.

Binding values are `none`, `history_previous`, `history_next`, `send:<bytes>`, or `send_cancel:<bytes>`. Escape notation accepts `\e`, `\r`, `\n`, `\t`, `\\`, `\xNN`, and `\^X` for an ASCII control character.

INI diagnostics and syntax limits

Boot continues after recoverable line errors and reports `LOADED WITH ERRORS (n)`. The first ten bad lines are listed in this form:

```
42: [bank3]: unknown_setting=value
```

Line numbers are one-based and right-aligned in four positions. The report contains the section and source line, not an elaborate error explanation. ANSI boot highlights each detail row with red paper; VT52 uses a `*` prefix. The total can exceed the ten retained details. The same report is written to `A0:7XTGVTRM.ERR` when storage permits, with CRLF and trailing `Ctrl+Z` (0x1A). A clean INI load removes a stale ERR file. If no INI is loaded, the banner reports `COMPILED DEFAULTS`. Application banks are neutral in that case; no application key map, launch/exit/clear command or BASIC adapter is enabled automatically.

Use one setting per line and comments on separate lines. The line buffer holds 191 characters excluding its terminator; longer lines are diagnosed, not silently accepted. Physical line breaks count even in comments. `Ctrl+Z` (0x1A) ends the logical INI, so anything after it is not parsed.

For Boolean settings prefer `true` / `false`. The parser also accepts lowercase `yes` / `no` and `1` / `0`; it does **not** accept `on` / `off`. Ordinary setting names and enum values are case-sensitive; use the lowercase spellings below. Key-chord names are case-insensitive. Global, palette and display section recognition is case-insensitive, but keep `[bankN]` lowercase.

Duplicate explicit key bindings are errors; ordinary scalar settings are not covered by that duplicate check. This is syntax validation, not a complete logical validator: it cannot prove an application understands configured bytes or that host shortcuts will reach the firmware.

Keyboard overrides in 7XTGVTRM.INI

A profile translates host key events into bytes or terminal actions. The guest application must understand those bytes. The profile does not install or discover the application’s key map: test the actual host keyboard/bridge and guest app. Turbo’s TINST-defined controls, TE’s installed controls and the current FAND mappings remain the reference for those specific installations.

Independent choices

Setting	What it changes
`input` Values: `custom`, `ansi`, `vt52`, `wordstar`	Shared default navigation bindings. It does not select an application, output mode, adapter, commands, prompt tracking or history timing.
`mode` Values: `ansi`, `vt52`	Interpretation of guest output. A VT52 output application can still receive explicitly configured WordStar-style input.
`input_adapter` Values: `none`, `mbasic_native`, `local_compose`	Local handling of editing keys and, for the BASIC adapters, MBASIC EDIT. This can consume a key instead of sending its configured bytes.
`screen_line`, `trim_left`, prefixes, numbered-line activation and history clear/delay	How the terminal tracks and retypes the application's visible input line. A preset does not enable these settings.
`[global] ccp_bank`	Which profile handles a recognized CP/M shell prompt; no application role is inferred from a slot number or name.

`## RAW` temporarily bypasses ordinary profile translation and adapter handling; it is not another `input` or output-emulation preset. Established terminal control commands remain available; see the RAW shortcuts in the User Guide.

Loading order and key priority

Configuration loading proceeds as follows:

1. Initialize every bank with the same neutral settings. Navigation bindings are inactive, `input=custom`, `input_adapter=none`, and application commands and prompt/retype settings are empty or disabled.
2. Parse the INI. Record explicit global and bank bindings, including explicit `none`, and each bank's selected preset and editing settings.

3. After the entire INI is parsed, expand each selected non-custom preset into that bank's navigation bindings. Explicit bank bindings are retained.
4. Publish the completed configuration for runtime use.

Putting `input=ansi` before or after `left=send:…` makes no difference. A preset fills missing bindings; it does not replace an explicit bank binding. Duplicate explicit definitions of the same key in the same scope are diagnosed, and the first accepted binding remains. Ordinary scalar settings such as `input` use the last accepted value. Section ordering does not change binding priority.

For an ordinary profile key, look up the binding in this order:

1. An explicit binding in the effective `[bankN]` section.
2. An explicit binding under `[global]`.
3. That bank's selected preset, or its neutral navigation binding.
4. For keys outside the navigation table, normal terminal/host fallback handling when no binding exists. Ordinary **Backspace**, text, **Enter**, main-row digits and keypad decimal have their normal handling; a preset does not redefine them.

Explicit `none` wins the binding lookup just like `send:…`. For navigation keys, an inactive binding suppresses the host's fallback sequence; it does not silently pass an ANSI arrow to CP/M. `input=custom` does not mean RAW input. Bank sections do not inherit one another, and names do not select presets.

Binding lookup is separate from executing the key. Reserved terminal controls such as forced return and viewport pan have their own routing before ordinary profile bindings. An enabled editing adapter can consume ordinary editing keys before their resolved binding is sent. Thus `left=send:…`, or `left=none`, does not disable a local composer's **Left** key handling. Use `input_adapter=none` when ordinary key-to-byte routing is wanted. The explicitly assigned CCP profile also uses native in-place **Home** / **End** handling for a send binding at a live CCP prompt; an explicit `home=none` or `end=none` remains inactive there.

Exact preset coverage

`input` value	Coverage before explicit overrides
`custom` (also when omitted)	No preset. Missing navigation bindings are inactive in every bank.
`ansi`	30 bindings: ten ordinary navigation keys, ten Ctrl variants, eight Alt variants, and Alt Up / Alt Down history.
`vt52`	Six bindings: four ordinary arrows and Alt Up / Alt Down history. Other navigation keys are inactive.
`wordstar`	12 bindings: ten ordinary navigation keys and Alt Up / Alt Down history. Ctrl/Alt navigation variants are inactive except Alt Up / Alt Down.

The following cells give the bytes using the same escapes accepted after `send:` in INI. `none` means inactive; it can be replaced by an explicit binding.

Key	`ansi`	`vt52`	`wordstar`
`up`	`\e[A`	`\eA`	`\^E`
`down`	`\e[B`	`\eB`	`\^X`
`left`	`\e[D`	`\eD`	`\^S`
`right`	`\e[C`	`\eC`	`\^D`
`home`	`\e[H`	`none`	`^Q\^S`
`end`	`\e[F`	`none`	`^Q\^D`
`page_up`	`\e[5~`	`none`	`\^R`
`page_down`	`\e[6~`	`none`	`\^C`
`insert`	`\e[2~`	`none`	`\^V`
`delete`	`\e[3~`	`none`	`\^G`
`up_alt`	`history_previous` (all three presets)		
`down_alt`	`history_next` (all three presets)		

ANSI's modifier variants are exactly:

Base key	Ctrl variant (`_ctrl`)	Alt variant (`_alt`)
`up`	`\e[1;5A`	History previous, as above
`down`	`\e[1;5B`	History next, as above
`left`	`\e[1;5D`	`\e[1;3D`
`right`	`\e[1;5C`	`\e[1;3C`
`home`	`\e[1;5H`	`\e[1;3H`
`end`	`\e[1;5F`	`\e[1;3F`
`page_up`	`\e[5;5~`	`\e[5;3~`
`page_down`	`\e[6;5~`	`\e[6;3~`
`insert`	`\e[2;5~`	`\e[2;3~`
`delete`	`\e[3;5~`	`\e[3;3~`

Presets do not supply Shift variants, letter/function-key macros, or Backspace bindings. Ctrl / Alt variants of the navigation keys not listed by a preset are inactive unless configured explicitly. Other supported chords can use their normal host fallback; reserved terminal shortcuts still have their own routing. There are no `input=mbasic`, `bbc`, `comal`, `adm3a`, `mz821` or `fand` presets. A protocol/style preset is shared only by profiles which explicitly select it.

Presets versus the tested working profiles

At the 261009 review, all ten working INI banks omit `input`, so all use `custom`. The earlier numbered application defaults were a separate mechanism, now removed. An explicit binding equal to a preset is valid and useful as a tested, visible mapping; it is simply redundant if that preset is later selected. Selecting a preset alone leaves every explicit binding in place.

Working profile	Relationship to the shared presets
CP/M and GENERAL	Keep CCP controls explicit; their Ctrl navigation differs from ANSI.
MBASIC	Uses `input_adapter=local_compose`. Cursor/editing behavior is adapter-owned; an ANSI preset would not replace that adapter.
BBC BASIC	ANSI can supply modifier fallbacks; preserve BBC cursor/edit controls and history clear/retype settings explicitly.
COMAL	The current ANSI template matches `input=ansi`; actual COMAL installation and guest acceptance remain pending.
Turbo Pascal	Keep the TINST-defined private 8-bit controls explicit. ANSI matches only shared fallback bindings; it does not configure TINST or replace its map. The current Alt Left / Alt Right fallbacks match ANSI. See the installed action tables.
FAND	All 12 active WordStar preset bindings match the configured bytes/actions. Retain its additional Ctrl / Shift navigation and save macro explicitly. This is a comparison of the current map, not proof of every FAND version’s requirements.
TE	Nine of the 12 active WordStar bindings match. Home, End and Insert differ; retain those exceptions and the four explicit Ctrl bindings.
ANSI APP	The current navigation map matches `input=ansi`.
DDT	Keep its limited explicit controls; no application preset is inferred.

TE's configured Home is bytes `11 53`: `Ctrl-Q` then **printable** S. The generic WordStar preset sends `11 13`: `Ctrl-Q` then `Ctrl-S`. End similarly differs as `11 44` versus `11 04`. These are not interchangeable encodings. TE also explicitly disables Insert while the generic preset sends `Ctrl-V`. The installed TE help and tested keyboard behavior take precedence over a generic style name.

For example, a TE section could explicitly select the shared preset while keeping these exceptions and the application-specific Ctrl bindings (other settings in the complete section still apply):

```
[bank7]
name=TE (wordstar)
input=wordstar
input_adapter=none
editing_surface=screen
home=send:\x11S
end=send:\x11D
insert=none
left_ctrl=send:\x01
right_ctrl=send:\x06
home_ctrl=send:\x11R
end_ctrl=send:\x11C
```

This is an optional equivalent key-map example, not a change to the working INI. FAND's 12 base/history bindings could likewise come from `input=wordstar`; its additional controls must remain explicit. The repeated ANSI modifier bindings in BBC/COMAL/ANSI APP, and the two Alt-arrow fallbacks in Turbo, can come from explicit `input=ansi` selections if that cleanup is chosen. Keep original tested exceptions when consolidating; do not remove a line merely because its app is called WordStar-like or ANSI.

The current bank controls and the installed guest configuration form a pair. RunCPM's `console_bits` setting controls console output masking; it is not what enables the private high-bit input bytes. The tested input path retains bit 7. See the compact key scheme and test the chosen host bridge's actual chords before treating a mapping as accepted.

```
[global]
kp_decimal=send:.
f5_ctrl_shift=send:\eQ

[bank3]
f5_ctrl_shift=none
num1_shift=send:1
```

The left side is `<key>[_ctrl][_shift][_alt]`, with the base key first and modifier suffixes in exactly that order. Use `f5_ctrl_shift_alt`, not `f5_alt_ctrl_shift`; names are case-insensitive. `_alt` is intended for Alt. RAlt/AltGr is reserved for the Czech layout's symbol entry, and is not a separate INI modifier.

Key group	Available base names
Cursor and editing	`left`, `right`, `up`, `down`, `home`, `end`, `insert`, `delete`, `page_up`, `page_down`
Common keys	`backspace`, `escape`, `enter`, `space`, `f1` … `f12`
Main number row	`num0` … `num9` (separate from the keypad)
Numeric keypad	`kp_decimal`, `kp_delete` (the latter is NumLock-off Delete)
Czech physical positions	`bs2`, `bs1` before Backspace; `p1`, `p2` after P; `l1`, `l2`, `l3` after L; `z1` before bottom-row Y / Z; `m1`, `m2`, `m3` after M
Letters	`a` … `z`, only with Ctrl or Alt; plain and Shift-only letters remain normal typing

Any subset of the three modifier words is syntactically accepted for the non-letter names. Plain and Shift-only `bs1` and `l3` are deliberately excluded because these are dead-accent positions on the current Czech keyboard: their normal composition must continue to work. `num1` … `num0` send ASCII digits by default even when the unshifted Czech keycap produces `+ěščřžýáíé`; `kp_decimal` sends `.` by default even when the host emits `,`. Shifted main-row digits are separate names such as `num1_shift`.

The Windows Mintty launcher supplies these number-row and decimal events through its per-session physical-key helper. Rebuild the selected desktop target and open a new mirror to load it. Keypad decimal and brightness controls have native acceptance; the corrected plain Czech number-row detection still needs a native retest. It recognizes the Czech number row and the NumLock-on decimal key without converting ordinary commas or pasted text. See Tooling Guide for setup and the keyboard probe. Plain/Shift row conversion for Linux Konsole/Kitty and Windows Mintty is selected by `XTGV_HACK_TOPROW` in host `tools/terminal.conf`; its default is `+ěščřžýáié`, and empty disables it. This host option is separate from application INI bindings and Alt-bank selection. Changes apply when opening a new mirror through the project launchers; see the Tooling Guide for configuration precedence and layout-specific limits.

The right side has five forms:

Value	Effect
`send:<bytes>`	Send the specified bytes in ordinary routing; adapter/CCP handling follows the rules above.
`send_cancel:<bytes>`	In ordinary routing, send bytes and discard the current terminal history line.
`history_previous` / `history_next`	Browse the effective history bank.
`none`	Suppress ordinary mapping/fallback. Adapter-owned editing keys follow the adapter routing described above.

Encoded bytes and control characters

- A key binding holds up to **16 decoded bytes**.
- `\e` is Escape, `\r` Return, `\n` LF, `\t` Tab and `\\` a backslash. `\xNN` inserts one hexadecimal byte.
- `\^K` sends `Ctrl-K` (0B). The backslash-caret consumes exactly one character: `\^K\^D` sends two controls; `\^K^D` sends `Ctrl-K` then printable `^D`. Bare `^K` stays printable.

Control notation	Decoded byte
`\^@`	00 (NUL)
`\^A` through `\^Z`	01 through 1A; lowercase letters also accepted
`\^[`	1B (Escape); `\e` is usually clearer
`\^\`, `\^]`, `\^^`, `\^_`	1C, 1D, 1E, 1F respectively
`\^?`	7F (DEL)

- Invalid forms such as `\^5` are rejected.
- Put comments on separate lines beginning with `;` or `#`. After a binding value, comment text would instead become part of that value.
- Duplicate explicit bindings report errors; the first accepted one remains. The sparse override pool holds 128 entries across global/bank scopes; the fixed per-bank legacy key slots do not consume that pool.

The current source supports both control notation and full-byte `\xNN` values. For example `send:\x81` encodes one byte, not a UTF-8 character. Use `console_bits=8` for an application expecting bytes above 7F.

INI recognition versus host delivery

- A valid binding does not guarantee that the host delivers its physical chord. A desktop shortcut can consume it first.
- Supplied Konsole/Kitty keymaps translate combinations with known safe fallbacks. Terminal-owned shortcuts, ambiguous AltGr combinations, unmodified dead accents and some punctuation combinations are not freely assignable application keys.
- Konsole’s key table does not explicitly distinguish left and right Alt.
- Ordinary unbound keys retain host output; pasted text is not remapped.

Reloading and persistence

`7XTGVTRM.INI` accepts CR, LF or CR/LF lines and treats `Ctrl+Z` (0x1A) (`1Ah`) as logical end of file. It can be edited with a CP/M editor and reloaded by `REBOOT`. Palette and brightness selections are saved lazily; ordinary keyboard zoom, pan and `VTZOOM` are temporary and do **not** rewrite boot `view_zoom`. Other settings and other display sections remain user-authored. The host transfer tool's deliberate `--cpm-text` option writes CR/LF plus `Ctrl+Z` (0x1A). Do not edit checksummed history files from CP/M. The INI cache and save transaction are described in the Technical Reference Guide.

Global settings

Setting	Accepted value / effect
`mode`	`ansi` (default) or `vt52`; bank `mode` may override it.
`console_bits`	`7` or `8`; compiled default is 7. Current device INIs may choose 8.
`ccp_bank`	`0` through `9`, or `none`. Explicit CP/M shell profile slot; omitted means `none`. Enables automatic effective-profile routing at a recognized CCP prompt and native CCP recall when that slot is selected.
`sdfat_spi`	Integer 1—60, requested SPI MHz for direct-SdFat boards. Startup/config reads use 20 MHz; missing/unreadable/invalid setting falls back to 20. Applied at boot after config/history handles close. Has no physical SPI effect on desktop host storage. Not a measured clock or reliability guarantee; the global value follows the card between boards.
`mode_palette`	0 through 9; default 0. Initial active palette, persisted after selection.
`cursor_blink_ms`	1 through 5000 milliseconds; default 500.
`backlight_level`	1 through 31; default 16. Last non-off level. Legacy 32 is accepted as 31. Runtime off is not persisted as zero.
`crt_rampup_ms`	0 through 60000 milliseconds; boot-only backlight warm-up, default 0 (disabled).
`exit_key`, `clear_key`	`none`, `escape_alt`, or `backspace_alt`. Defaults: `exit_key=escape_alt`, `clear_key=backspace_alt`.
`viewport_gutter_rgb`	Decimal `R,G,B`, each 0..255, or `palette`. Default: inherit palette.

Setting	Accepted value / effect
`viewport_active_gutter_rgb`	Decimal `R,G,B`, `active_line`, or `palette`. Default: inherit palette's active-gutter policy.
Key override	Any supported chord and binding described above, shared across banks.

These settings select the logical events sent by the supplied Alt Q and Alt Backspace host mappings. Either event can select exit or clear; `none` disables the corresponding policy entry. They are processed before ordinary profile bindings, not arbitrary keyboard-chord remaps.

Legacy `lalt+ralt+escape` and `lalt+ralt+delete` remain accepted aliases for `escape_alt` and `backspace_alt`, respectively. They do not require AltGr; use the new spellings in new INIs. Older firmware needs rebuilding before it can read the new spellings. There is no implemented `boot_ansi` setting; adding that earlier proposed name causes a load error.

After the first themed frame is drawn/flushed, `crt_rampup_ms` gradually raises the backlight from OFF to `backlight_level` (default 16). It applies to all palettes, not only GREEN/AMBER. For example:

```
[global]
crt_rampup_ms=1500
```

- There are 32 logical states: OFF=0, intermediate levels 1..30, full ON=31. Full ON means the board's configured maximum, not necessarily 100% PWM duty.
- Each HAL maps levels to its own duty values. Level 1 is its intended lowest usable visible level, not a common brightness percentage across panels.
- The ramp traverses logical levels over the configured duration: a stepped effect, not a calibrated linear-light fade.

Dated visual tuning and deployment evidence remains in RP2_2 notes and RP2_147 notes. The configured backlight step chords stay within 1..31; from OFF they first restore remembered brightness. Explicit brightness controls cancel an active ramp immediately. Transient ramp levels never overwrite the saved target. Private `CSI >63;32v` remains a legacy alias for full ON; 33/34 still mean increase/decrease. No new INI setting is needed for ordinary immediate startup.

Display and per-bank geometry

Use `[display:HAL_ID]` for the matching compiled display-profile identifier. JC8048W550 uses `[display:ES3_5]`; JC8048W535 uses `[display:ES3_35]`. Check `hardware/arduino_7xtgv/boards/` for each target's constants; `src/terminal/display_policy.h` exposes the selected policy. Other display sections do not apply their settings, allowing a shared card. Basic line syntax and line-length errors can still be reported in an inactive section.

These are **compiled fallbacks**, not a report of the currently inserted card's INI. `VTINFO` shows the running target and resolved presets. The logical maximum is the backing-store limit, not the visible TFT width; odd configured columns round down to even.

Exact target	Display ID	Panel px	Logical max
`jc8048w550`	`ES3_5`	800x480	133x48
`jc8048w535`	`ES3_35`	480x320	80x32
`rp2350-touch-lcd-2`	`RP2_2`	320x240	80x24
`rp2350-lcd-1-47-a`	`RP2_147`	320x172	80x24
`jc8012p4a1c`	`EP4_10`	1280x800	213x80
`esp32c6-lcd-1-47`	`EC6_147`	320x172	80x24

Compiled logical geometry for each zoom preset:

Display ID	1x	2x	3x	4x
`ES3_5`	132x48	80x24	44x16	32x12
`ES3_35`	80x32	40x16	26x10	20x8
`RP2_2`	80x24	40x12	26x8	20x6
`RP2_147`	80x16	40x8	26x5	20x4
`EP4_10`	212x80	106x40	70x26	52x20
`EC6_147`	80x16	40x8	26x5	20x4

- Compiled boot zoom is 2 for EP4_10 and 1 for the other five boards.
- Compiled `render_flush_hz` is 30 for EP4_10, ES3_35 and ES3_5; 0 for RP2_2, RP2_147 and EC6_147. An INI value such as 10 overrides the fallback. This is a software scheduling policy, not a measured display refresh rate.

These are local display-profile/group IDs, at most eight characters with optional letter variants (for example `ES3_5A`). They are distinct from exact build/upload targets such as `jc8048w550`. Two HALs using the same ID share that INI section; use different IDs when settings should be independent. Sharing is intentional for compatible HALs; a different pixel resolution gets a new short ID. Current IDs are compiled in the HAL policy, so editing a host JSON label alone does not rename a firmware selector. Host target data records both names and tools display them together.

Setting	Meaning
`view_zoom`	1..4, the boot scale for 6x10 glyph cells. Temporary ZOOM/keyboard changes do not rewrite it.
`zoom1_terminal` through `zoom4_terminal`	Current logical `COLSxROWS` presets. The boot scale selects one; `VTZOOM n` deliberately selects another. Ordinary keyboard zoom leaves the logical size alone. Odd column counts round down to even. Both dimensions must fit the HAL's logical backing limits.
`render_flush_hz`	0..60; 0 removes the software flush interval. Linux/Windows desktops finish dirty visible cells in one pass and publish complete changed frames at this rate, including simulated direct-display targets. This is not the physical panel refresh rate.
`rgb_refresh_hz`	Accepted compatibility placeholder only; physical RGB timing remains compile-time.

- A bank can override `zoom1_terminal` through `zoom4_terminal`. These presets select logical dimensions together with a deliberate preset selection; ordinary keyboard zoom remains independent of logical geometry.
- Logical width/height may exceed the physical view. Ctrl Shift with an arrow key pans by axis; Ctrl Shift Home / Ctrl Shift End scan views in row-major order. Positions include both exact edges and are recalculated for zoom and geometry. The mirror still represents the complete logical terminal.
- Bank overrides are shared across displays: an oversized override is reported and rejected on a smaller HAL, which retains its display geometry. Omitted settings retain that HAL's defaults, even with a shared display-profile ID.
- Brightness, palette selection and CRT ramp time remain global, not per-display.

See selection, persistence and identity audit for the complete source-grounded flow and limitations.

Additional bank settings

All slots use the following same fallbacks. There are no per-application or bank-number exceptions. Configure non-neutral application behavior explicitly.

Setting	When omitted	Meaning / limit
`name`	`Bank N`	Displayed profile name; stored up to 20 characters. It does not select behavior.
`input`	`custom`	Shared key-map preset: `custom`, `ansi`, `vt52` or `wordstar`; see exact mappings.
`mode`	Global mode	Optional `ansi` or `vt52` output interpretation override.
`zoom1_terminal` through `zoom4_terminal`	Display geometry	Optional per-bank geometry; validated against the selected HAL, as above.
`on_select`	Empty	Up to 64 decoded bytes; emitted only at live CCP on selection.
`exit_command`, `clear_command`	Empty	Up to 64 decoded bytes each; no `send:` prefix. Empty means no application command.
`command_char_delay_ms`	30	0..1000 milliseconds between generated exit command characters; 0 sends the command as a burst. Clear commands are not paced. See the timing note below.
`history_clear`	Empty	Up to 16 decoded bytes used before retyping recalled text; otherwise the runtime uses its ordinary erase/recall transport.
`history_retype_delay_ms`	0	0..1000 milliseconds between clear and retyping.
`line_prefix`, `activate_prefix`	Empty	Up to 8 decoded bytes each for screen-line tracking/activation. Empty activation prefix needs no explicit prompt to activate tracking.

Setting	When omitted	Meaning / limit
`screen_line`	`false`	Use the displayed input line for history tracking, rather than only the typed line model.
`trim_left`	`false`	Trim leading whitespace in the tracked screen line.
`activate_numbered_line`	`false`	Allow numbered BASIC lines to activate tracking.
`input_adapter`	`none`	`none`, `mbasic_native` or `local_compose`; select application editing handling explicitly.
`command_new_line`	`false`	Issue a newline before the configured profile command.
`escape_cancels`	`false`	Cancel the captured history line on Esc.
`active_line`	`true`	Enable the active cursor-row highlight.
`active_line_rgb`, `cursor_rgb`	Active palette	Decimal `R,G,B` override, or `palette` to inherit the current theme.
`editing_surface`	`line`	`line` or `screen`; parsed/stored, but not yet used to select a distinct recall transport. Not an implemented completion-list switch.

Generated exit commands wait for prior input to be consumed before starting each character's delay. They do not catch up in a burst after a slow guest or display frame. The delay also applies before the first command character, after any erase/newline prefix; a longer configured clear/retype delay is retained. Profile/session changes cancel a pending command. Clear commands are sent as a burst, retaining any configured initial clear/retype delay. This keeps long MBASIC clear expressions responsive. This pacing does not alter ordinary typing, keyboard escape sequences, terminal replies, history recall or `on_select`. Thirty milliseconds is a minimum, not an exact delivery interval: guest consumption and terminal polling add time (about 50 ms per character in the tested browser at 60 Hz). It is a tested host starting point, not a timing guarantee for every guest and board; increase the profile value, for example to 100, if a slower guest loses input.

Tab is not currently an INI base-key name. Ordinary Tab can still pass as input. Do not infer new key names or automatic private 0x80..0xFF mappings from the core-key design notes; assign supported chords explicitly with `send:\xNN`.

Palette definitions and selection

`[palette0]` through `[palette9]` are in this same INI. Each slot starts from built-in PAPER defaults independently; missing entries in palette 1 do not inherit edited palette 0. `VTPAL n` selects a slot and persists `mode_palette`. Editing the RGB definitions still requires reloading at boot. A theme change repaints text/UI and can retire a retained graphics image; xterm256 plot RGB conversion itself does not use these editable ink slots.

Palette key	Value
`paper0` through `paper3`	Decimal `R,G,B`, each component 0..255.
`ink00` through `ink15`	Same RGB syntax; ink names require two digits.
`active_line_rgb`, `cursor_rgb`, `viewport_gutter_rgb`	Same RGB syntax; additional UI roles, not extra native papers.
`viewport_active_gutter_rgb`	RGB or `active_line`.

Use `palette` on bank active-line/cursor and global gutter keys to inherit; explicit RGB overrides win. `RGB565`, hexadecimal color literals and symbolic color names are not accepted INI color values.

Current configured presets are 0 PAPER, 1 GREEN, 2 AMBER, 3 BW, 4 PLASMA, 5 VFD.

For ANSI/SGR slot selection and text attributes, see Text attributes and color interpretation. The palette comparison example is in Appendix A.

5. Terminal interfaces

Output interpretation modes

There are two terminal-output interpretations: **ANSI/VT100** (the normal default, including implemented xterm colour extensions) and **VT52**.

- Guest commands `VT100` / `VT52` select ANSI/VT100 / VT52 on all routes.
- In the native mirror, Ctrl] then 1 / Ctrl] then 2 are prefix alternatives.
- Native bridge options `--mode ansi` / `--mode vt52` select one at connection.
- Applications use `CSI ?2h` for ANSI, `CSI ?2l` for VT52, and VT52 `ESC <` to return to ANSI. These transitions do not save a new INI boot mode.

`## RAW` is only an input-routing state, application profiles are keyboard translation choices, and GCLS/xterm256 plot is a temporary graphics canvas; none is a third persistent terminal emulation mode. Application cursor/keypad flags are likewise negotiated submodes within ANSI/VT100, not separate terminal types.

Implemented escape-sequence subset

ANSI/VT100-style output supports CR, LF/VT/FF, BS, BEL, SI/SO and tabs at eight-column stops; ESC H sets a stop and CSI g clears one or all stops.

- **Position and erase:** CSI A/B/C/D, E/F, G, H/f and d move/address the cursor. CSI J/K erase modes 0/1/2 work; CSI 3 J does not clear scrollback because there is none.
- **Scrolling:** ESC D/M/E index/reverse-index/next-line; CSI r sets top/bottom margins and CSI L/M inserts/deletes lines. Margins span complete logical rows, not pixel windows or left/right-bounded rectangles.
- **State:** DEC private ?6, ?7 and ?25 control origin, autowrap and cursor visibility. ESC 7/8 and CSI s/u save/restore the relevant cursor, rendition and charset state.

- **Modes and queries:** CSI ?1 and ESC =/> select cursor/keypad application modes. ESC #8, ESC c, CSI 5n/6n, CSI c and ESC Z have their implemented alignment/reset/status/identity meanings. CSI ?2 l enters VT52.

G0/G1 may designate ASCII (ESC (B / ESC) B) or DEC Special Graphics (ESC (0 / ESC) 0); SI/SO selects the active set. The DEC test notes below identify approximations and unsupported control pictures. SGR supports 0, 1/22, 4/24, 5/25, 7/27, 30..37/39, 40..47/49, 90..97 and 100..107, subject to the palette/paper mapping below. This is not full ANSI or VT100 conformance.

VT52 mode supports ESC A/B/C/D, H, I, J, K; ESC Y plus row+32 and column+32; ESC Z identification, ESC F/G graphics/ASCII, ESC =/> keypad selection and ESC < return to ANSI. Its graphics use the implemented DEC set, not the original VT52 character ROM.

There is no alternate screen, scrollback, 132-column mode, double-sized lines, mouse, UTF-8 guest input, automatic pan or binary console transport. Unsupported CSI commands are ignored as units; string controls are consumed without display. Bytes above 127 do not directly index the 6x10 font. The private 7xtgv-terminal query/control protocol is catalogued in the Technical Reference Guide.

Text attributes and color interpretation

The slots in this mapping are configured under Palette definitions and selection.

ANSI color	ANSI name	Normal ink slot	ALT ink slot
0	Black / default ink	00	08
1	Red	01	09
2	Green	02	10
3	Yellow	07	15
4	Blue	03	11
5	Magenta	06	14
6	Cyan	05	13
7	White / silver	04	12

SGR 30..37 selects the normal set; 90..97 selects the ALT set. SGR 1 (bold) selects the ALT variant by bit 3, not a thicker glyph. It need not be brighter: the palette chooses the contrast appropriate to paper or dark backgrounds. SGR 22 clears bold; SGR 0 resets rendition. Other supported text attributes include underline (4/24), blink (5/25) and reverse (7/27).

ANSI backgrounds 40..47 map to paper slots `2,1,2,2,3,1,2,0` respectively; 100..107 use the same mapping. In particular ANSI black background does not mean a literal black RGB paper in this compact text renderer. SGR 49 resets to paper0. This four-paper policy is intentional current behavior, not unrestricted ANSI background-color fidelity or the future four-plane native specification. Indexed SGR 38/48 colors operate on the active plot canvas, not as arbitrary 256-color text cells. Truecolor SGR sequences are consumed but not applied.

Internal ink order is implicit, red, green, blue, shade, cyan, magenta, yellow (`irgbscmy`), with ALT slots 8..15 (`IRGBSCMY`). This does not change ANSI order.

Terminal control and query map

Interface	Reference
Host keyboard and prefix controls	Global terminal keys; controls are routed by the terminal before ordinary application bindings.
Guest CP/M helpers	Shared A0 commands; command arguments and effects are defined in Part 3.
Guest output sequences	Implemented escape-sequence subset; output interpretation and text attributes are defined in this part.
Private requests and replies	TRG: Private protocol boundaries; byte formats, routing and ownership are developer interfaces.

Use the named command or control references for syntax. A host shortcut, a +guest helper and a private escape request can expose related operations, +but each enters through its own documented interface.

Graphics extension xterm256 plotting

In the native mirror, Ctrl] then g or Alt Shift Backspace starts a local GCLS graphics layer. In the browser, use the guest plotting scripts which initialize graphics themselves; native prefix shortcuts do not apply. The extended private form is `CSI >4;zoom;x;y v`.

- **Coordinates:** `zoom` is 1..4; `x,y` are physical pixels from the display's top-left corner, not logical cells and not multiplied by zoom. The origin does not move with the gutter or viewport; x=0 may cover the gutter.
- **Samples:** each source sample becomes a `zoom × zoom` RGB565 block, clipped at the physical client boundary. Indexed-color samples (`CSI 38;5;<index>m` plus byte values) draw on the TFT, SDL panel or browser canvas; they are not pixels in the native text mirror or browser screen-text snapshot.
- **Lifetime:** the cell layer remains logically present; opaque text can paint over the image. The first later terminal scroll retires the plot and restores the gutter without a full-screen clear.

The direct plot ends on SGR reset (`CSI 0m`). Rows used by status text and the active cursor are then repainted from terminal memory, including when the plot was launched by `SUBMIT`; the remaining graphics canvas stays visible. A normal `CLS` clears the terminal cells, while the native mirror prefix Ctrl] then r explicitly returns the screen to the normal terminal surface.

For bundled programs and launch recipes, see Graphics and timing helpers and Plotting demonstrations in Appendix A.

Source references and implementation boundaries

Source, protocol, upload safety and licensing are maintained in the Technical Reference Guide.

Appendix A. Diagnostics, tests and demonstrations

These optional checks and examples exercise the interfaces described in Parts 3 and 5. Dated records retain measurements and hardware acceptance.

CARDMARK raw SD read diagnostic

`CARDMARK [CSD] [sectors [MHz [start-sector]]]` requires matching firmware on our direct-SdFat/SPI targets: ES3_35, ES3_5, EP4_10, RP2_147, RP2_2 and experimental EC6_147. Desktop reports this raw-card diagnostic as unsupported. The former W535 SD_MMC alternate has been removed; all six current physical targets use SPI/SdFat. Defaults: 1024 sectors, the configured operating SPI request, and start sector 0. Sectors are 512 bytes; count range is 1..1048576 and requested clock range is 1..60 MHz. Example: `CARDMARK 4096 50`.

Add `CSD` anywhere among the arguments for named CSD fields, for example `CARDMARK 4096 50 CSD`. Matching is case-insensitive. The compact report orders raw CID, decoded identity, raw CSD, capacity in whole MiB (rounded down, with exact 512-byte sector count), and encoded speed. CMD6 checks High Speed support where advertised; it does not select High Speed mode. An unsuccessful query reports unknown support and does not prevent the raw read test.

Detailed decoding covers SD CSD v1.0 (SDSC) and v2.0 (SDHC/SDXC). CSD v3.0 (SDUC) is identified but not decoded or supported by this SdFat backend. These are SD layouts, not MMC/eMMC layouts. See register decoding notes.

The native service reads reference data at requested 20 MHz and compares three trial reads per batch at the selected clock. It reports card CID/CSD, capacity, compared bytes and raw-read throughput, then restores the operating clock. The ` … ` hint appears before test reads and the result follows on that line. Ctrl Alt C cancels in the integrated terminal build. It runs in ordinary writable firmware too; CARDMARK itself uses only read operations. If clock restoration fails, storage is stopped and a reset is required rather than returning to CP/M.

Clock values are requested, not measured. A PASS demonstrates read agreement for the selected range, not write reliability or complete card qualification. Use a backed-up/spare card for experimental clocks. Raw throughput is not the same measurement as VTDRV's BDOS/file throughput. Detailed implementation and results: CARDMARK checkpoint.

Visual tests in 7xtgv(C1) and DEC graphics

Select `7xtgv(C1)` using `C1:`, start `MBASIC`, then `RUN "VTESTS"`. Save maintained sources with `SAVE "VTESTS",A` (ASCII); load with ordinary `LOAD "VTESTS"`.

The maintained local source has isolated attribute comparisons in item 3, VT52 movement/erase checks in item 5, a steady DEC frame in item 7, and five scrolling-region checks in item 9. Item 9 requires **80x24 logical geometry**, independently of the panel's visible viewport. Updated source must be deployed separately; the physical card may still hold an older menu. See test scope. `VTPAL.BAS` is also maintained in `7xtgv(C1)`; its physical-card location may differ.

VTESTS item 4 has two 40x16 pages: a continuous frame with junctions, patterns, scanlines and G1/SO/SI selection, then all 32 DEC Special Graphics positions 0x5F..0x7E. The implementation supports G0/G1 designation and SI/SO switching; this is not a complete VT100 conformance claim. Six control pictures (`b`, `c`, `d`, `e`, `h`, `i`) currently fall back to question marks. TFT pi/pound use approximate QL p/L glyphs while the PC mirror uses Unicode equivalents. Historical W535 visual acceptance does not qualify every current board/build; physical retesting and exhaustive protocol/glyph checks remain separate work. See the complete inventory and test scope.

Palette comparison example

For complete example palettes and the comparison program see palette configuration notes and VTPAL.BAS comparisons. The maintained BASIC source is in `7xtgv(C1)`; inspect a live card before asserting which copy is installed there. `VTPAL.BAS` starts with the current palette and selects another only when a digit is pressed. With the MBASIC profile selected, its `exit_command=system\r` and paced command delivery let Alt Q return both `VTPAL.BAS` and `VTESTS.BAS` to MBASIC's OK prompt after terminal cleanup. The tools recognize the complete SYSTEM input, case-insensitively; an ordinary `s` is not an exit key. Their existing Q behavior remains unchanged. At the OK prompt, the same shortcut still exits MBASIC to CP/M. This needs the updated runtime and BASIC sources; it does not add a generic Alt+Q handler to arbitrary BASIC programs. VTPAL.BAS still has obsolete optional-level help for slots 3..6 and older INK/SILVER labels; that help is not the current preset definition.

Graphics and timing helpers in xterm256(C2)

Command	Purpose
`GCLS [zoom [x y]]`	Arm local xterm256 plotting without clearing or redrawing the screen. `zoom` is 1…4; `x,y` are unscaled physical pixel coordinates from the display's top-left corner, independent of terminal zoom, gutter, or pan. Bare `GCLS` retains `ESC[>4v` and defaults to 1× at `(0,0)`. Use `CLS` separately when wanted.
`SYNC`	Forces pending verified terminal-history and persistent settings saves immediately. Use before a long graphics stream or likely hardware reset.
`VTPAD [n]`	Prints `n` CR/LF pairs; default is 18. It provides visible space where SUBMIT would discard empty script records.
`VTSTART` / `VTSTOP`	Save and report elapsed RunCPM uptime around a test, using `VTIME.DAT` in the current DU.
`SUBMIT BROT_BS` / `SUBMIT BROT_PS`	Time the renderer with VTSTART/SYNC/GCLS/VTSTOP. BS plots at pixel (200,50); PS first pads eight lines and plots at (400,50). Neither script clears the screen; small panels may clip these fixed origins.
`SUBMIT VTPARG n`	Argument-expansion test: runs `VTPAD n`.

- Our helper sources and retained HEX/listings live in `7xtgv(C1)`. Build in a disposable copy, then explicitly install tested COM files in `cpm(A0)` or the demo area. For our Intel 8080 `.ASM` helpers use `ASM name`, then `LOAD name`; our `.Z80` helpers use SLR `Z80ASM name`.
- Imported Brot sources remain in `xterm256(C2)`, with `BROTINFO.TXT` attribution. Their suffixes do not imply compatibility with Digital Research ASM.

- The Brot names deliberately use underscores: TE treats `-` as an editing command separator. Use `BROT_BS` and `BROT_PS`, not hyphenated names.

Plotting demonstrations

- **Examples:** `BROT_BS` and `BROT_PS` in `xterm256(C2)` demonstrate the bare path.

Use the supplied submit scripts in `xterm256(C2)`:

```
C2>SUBMIT BROT_BS
C2>SUBMIT BROT_PS
```